Week 2 Workshop

Shell Tools and Scripting

Objectives

- To design `grep`, `find`, and text-processing pipelines that answer real questions about logs and files.
- To review and improve Bash scripts using arguments, conditionals, loops, safe quoting, and safety flags.
- To explain how SSH keys, `ssh`, `scp`, and remote commands fit into command-line workflows.

For all activities, work in groups of 3 – 4.

How to use this handout. This week has two group activities. The demos show each tool once; the activities ask your group to apply the tools to a *different* problem. For each task:

1. agree on what question you are answering;
2. design the command or script together;
3. be ready to defend each part of your answer.

Quoting, variables, and exit-status predictions happen live during the demos, so they are not repeated here.

If editing a `.sh` file is new to you, complete the separate *Week 2 Practical Guide: Editing Shell Scripts* before the workshop. You will need to revise a script in Activity 2.

Activity 1: Log Investigation

Scenario. A web server misbehaved overnight and all you have is its log and source tree. Your group's job is to answer concrete questions with `grep`, pipelines, and `find` — and to explain *why* each command is built the way it is.

Hint.

A **glob** is a filename pattern. In a glob, `*` matches any sequence of characters, `?` matches one character, and `[0-9]` matches one digit. For example, `*.log` means every filename ending in `.log`.

The shell expands an unquoted glob such as `ls *.log`. In `find . -name '*.log'`, keep the glob quoted so that `find`, rather than the shell, receives the pattern.

A. Interrogate the log. Suppose `server.log` contains:

```
INFO Startup complete
WARN Disk space low
WARNING CPU temperature high
ERROR 404 Page not found
ERROR 500 Database unavailable
ERROR 404 Page not found
```

Answer each question with one command or pipeline, and note what the output would be.

1. Which lines report errors? (Print lines beginning with `ERROR`.)
2. Which lines are warnings? (Match both `WARN` and `WARNING` with one pattern.)
3. **Which error code occurred most often?** Build a pipeline that counts each distinct code (such as `ERROR 404`), highest count first, then explain each stage in the table below.
4. Did anything `CRITICAL` happen? Use exit status only — no output.

Pipeline stage	**What this stage contributes**
`grep`	
`sort`	
`uniq -c`	
`sort -nr`	

B. Track down the files. Write one `find` command for each task.

1. Find all Python files under the current directory.
2. Find all shell scripts under the current directory.
3. Find every `.log` file below the current directory, including files in nested directories.
4. Search every Python file for `TODO` or `FIXME`, showing line numbers.

Activity 2: Script Review and Improvement

Focus. This is the main group task of the workshop — plan for about 25 minutes. Use this activity to recognise common Bash reliability problems and redesign a script so failures are visible. Everything from today — quoting, exit status, pipelines, safety flags — comes together here.

A. Review the fragile script.

```
#!/usr/bin/env bash
file=$1
if [ -f $file ]; then
    grep ERROR $file | sort | uniq -c
else
    echo File not found: $file
fi
```

Find at least five problems or risks. Consider spaces in filenames, missing arguments, standard error, exit status, pipeline failure, and quoting. *You do not need the exact flag names yet (Part C covers those) — describe each problem in plain language first.*

B. Safer script design. Rewrite the script so that it:

- uses Bash explicitly;
- fails on unset variables;
- fails when any pipeline stage fails;
- quotes variables safely;
- sends missing-file messages to standard error;
- exits with status `1` when the file does not exist.

C. Explain the safety flags. In your own words, explain what each line protects against.

Line	Protection
`#!/usr/bin/env bash`	
`set -u`	
`set -o pipefail`	
`file="$1"`	
`echo ... >&2`	

D. Loop design. Design a script that scans every `*.log` file in the current directory and prints the names of files containing `ERROR`. What should happen if no log files match?

Final Checkpoint Preparation

SSH key concepts are introduced in the seminar; you will generate and use a key in the Applied session. Notice where the public key goes, why that one is safe to copy, and why the private key must remain on your device.

Before the Poll Everywhere checkpoint, make sure your group can answer:

1. Why can `cd Week 2 Materials` fail even when the directory exists?
2. Why is `*.log` quoted in `find . -name '*.log'`?
3. Why does `sort` usually appear before `uniq -c`?
4. What does `set -o pipefail` change?
5. Why must a private SSH key stay private?

After the workshop: on Ed, read the worked `log-report` example and try the `warn-summary` coding challenge. You can run both directly on Ed or download the files and run them in your own terminal.